

METHODS AND PROCESSES FOR OPTIMIZATION OF CLOUD API PERFORMANCE THROUGH AI BASED MACHINE LEARNING ENSEMBLE CACHE MANAGEMENT

https://www.researchgate.net/profile/Kalyan-Chakravarthy-Thatikonda/publication/389521911_METHODS_AND_PROCESSES_FOR_OPTIMIZATION_OF_CLOUD_API_PERFORMANCE_THROUGH_AI_BASED_MACHINE_LEARNING_ENSEMBLE_CACHE_MANAGEMENT/links/67c6a5c896e7fb48b9d81623/METHODS-AND-PROCESSES-FOR-OPTIMIZATION-OF-CLOUD-API-PERFORMANCE-THROUGH-AI-BASED-MACHINE-LEARNING-ENSEMBLE-CACHE-MANAGEMENT.pdf

- **2025**

1. Qual é o problema de pesquisa que o artigo busca resolver?

Segundo o autor, estratégias tradicionais de cache apresentam limitações importantes em ambientes modernos com tráfego altamente dinâmico, múltiplos microsserviços e grande volume de requisições simultâneas.

O trabalho argumenta que esses modelos convencionais têm dificuldade para lidar com:

- picos de tráfego
- mudanças rápidas de padrão de uso
- dependências complexas entre serviços
- diferentes tipos de payload
- workloads variáveis

Isso pode gerar:

- aumento de latência
- baixa taxa de cache hit
- desperdício de memória
- sobrecarga nos servidores backend

2. Qual é o contexto ou domínio de aplicação do trabalho?

O trabalho está inserido no contexto de:

- cloud-native applications
- microsserviços
- APIs distribuídas
- sistemas em nuvem
- plataformas de alta escala

Além disso, o artigo testa o modelo em múltiplos provedores de nuvem:

- AWS
- Google Cloud Platform
- Azure

o que aproxima bastante o estudo de cenários corporativos reais.

3. Qual é a principal pergunta de pesquisa ou hipótese levantada pelos autores?

A principal hipótese do artigo é que técnicas de machine learning ensemble conseguem otimizar significativamente o gerenciamento de cache em APIs cloud-native quando comparadas a métodos tradicionais.

O autor defende que a combinação de múltiplos algoritmos de IA permite:

- prever padrões de requisição
- identificar relevância de cache
- melhorar políticas de substituição
- reduzir latência
- aumentar cache hit rate
- diminuir carga nos servidores

4. Quais são os métodos ou abordagens utilizadas (algoritmos, modelos, técnicas)?

O artigo propõe um modelo ensemble baseado na combinação de:

- Random Forest (RF)
- XGBoost
- Support Vector Machines (SVM)

Cada algoritmo possui uma função específica dentro do sistema:

- RF → reconhecimento de padrões de requisição
- XGBoost → análise temporal e previsão
- SVM → classificação de relevância de cache

Além disso, o sistema utiliza:

- feature engineering
- análise temporal
- scaling adaptativo
- data preprocessing
- anomaly handling

- deep learning principles
- adaptive scaling
- reuse distance analysis

O framework também trabalha com:

- logs históricos
- padrões de tráfego
- payloads
- dependências temporais
- métricas de uso em tempo real

5. Quais dados ou bases de dados foram utilizados no estudo?

O artigo não utiliza uma base pública específica.

Os modelos trabalham com:

- timestamps de requisições
- payloads de APIs
- métricas de cache hit/miss
- padrões históricos de tráfego
- dados temporais
- comportamento de usuários
- métricas de CPU/memória
- informações distribuídas entre regiões geográficas

O sistema processa milhões de requisições concorrentes em tempo real, utilizando pipelines contínuos de coleta e processamento de dados.

6. Como foi feito o desenho do experimento ou da avaliação?

Os experimentos foram realizados em um ambiente distribuído com:

- 128 nós
- múltiplos provedores cloud
- diferentes cenários de carga

As infraestruturas utilizadas incluíram:

- AWS r6g.8xlarge
- Google Cloud n2-standard-32
- Azure Easv4

Os testes compararam:

- cache tradicional LRU
- estratégias convencionais
- modelo proposto com IA ensemble

O sistema foi avaliado sob:

- tráfego intenso
- workloads variáveis
- múltiplos padrões de requisição
- cenários de pico

com coleta contínua de métricas a cada 50ms.

7. Quais métricas de avaliação foram utilizadas para medir os resultados?

As principais métricas utilizadas foram:

- cache hit rate
- p95 latency
- p99 latency
- mean response time
- throughput
- utilização de memória
- carga de servidor
- prediction accuracy
- adaptation speed
- model convergence
- reliability
- resource utilization efficiency

O artigo também mede:

- tempo de adaptação
- estabilidade
- precisão de classificação
- generalização do modelo
- escalabilidade do sistema

8. Quais foram os principais resultados e conclusões do artigo?

Os resultados mostraram melhorias bastante relevantes em comparação com métodos tradicionais de cache.

Entre os principais ganhos apresentados:

- cache hit rate passou de ~65% para ~83%
- redução significativa de latência p95 e p99
- redução de carga dos servidores backend
- menor uso de memória
- maior estabilidade sob tráfego intenso
- adaptação rápida a mudanças de padrão
- escalabilidade linear até milhões de requisições simultâneas

O artigo afirma que o modelo conseguiu:

- manter alta performance durante picos
- reduzir degradação sob carga
- melhorar eficiência operacional
- aumentar confiabilidade do sistema

Além disso, o sistema demonstrou comportamento consistente entre AWS, Azure e GCP.

9. Quais as limitações apontadas pelos autores?

O artigo reconhece algumas limitações importantes:

- aumento da complexidade de integração
- overhead de sincronização em edge computing
- delay de adaptação em mudanças bruscas de workload
- dificuldade de integração com infraestrutura existente
- necessidade de mais validação em setores específicos
- limitações na identificação de padrões completamente novos

O autor também aponta desafios relacionados a:

- segurança
- multi-tenant environments
- tuning automático
- cache coherency distribuído

10. Que oportunidades de pesquisa futura ou lacunas o artigo indica?

O trabalho sugere várias possibilidades futuras:

- melhoria dos modelos preditivos
- integração mais eficiente com edge computing
- tuning automático de parâmetros
- segurança para sistemas ML-based caching
- melhor adaptação em tempo real
- otimização em ambientes multi-tenant
- novos algoritmos de reconhecimento de padrões

11. O que esse artigo pode contribuir ou inspirar o seu trabalho?

De grande utilidade porque traz uma abordagem extremamente prática de otimização de performance usando IA aplicada diretamente à infraestrutura de APIs.

Enquanto outros trabalhos discutem:

- auto-scaling
- observabilidade
- arquitetura
- operações inteligentes

esse aqui entra com foco em cache management, enriquecendo a fundamentação teórica, mostrando que otimização de performance não acontece apenas em APIs, gateways e escalabilidade